Assumptions and design decisions

The assignment leaves several points open. This document records how each was interpreted, and the main design decisions with their reasoning.

- Scope and behaviour
- Interpreting questions
- Data and schema
- Safety
- LLM and cost
- Users, sessions and limits
- Engineering decisions
- Out of scope

Scope and behaviour

#	Assumption	Reasoning
S1	"SQL and database-related" includes questions **about the database itself** ("What does Segment mean?", "How are orders linked to customers?") and general SQL concepts, answered from the knowledge base	They are clearly database-related, and refusing them would be unhelpful
S2	Out-of-scope requests are refused with the **exact wording** from the brief: "*I'm designed to assist only with SQL and database-related tasks. Please ask a question related to the provided database schema.*"	The brief specifies the text
S3	Destructive requests (DELETE/UPDATE/INSERT/DROP/ALTER/TRUNCATE, or plain-English equivalents such as "remove all cancelled orders") get a **different**, polite refusal that offers a read-only `SELECT` of the affected rows	They are in the database domain, so the out-of-scope wording would be misleading. The brief forbids generating them, not discussing them
S4	Write SQL pasted for *explanation* ("Explain this: `DELETE FROM …` ") is still refused	The agent is strictly read-only; explaining write statements is not needed for the use case and keeps one simple rule
S5	Executing the query is **on by default** and can be switched off per request (the "Run queries" switch)	The brief lists execution as optional/bonus; showing results makes the answer verifiable
S6	When the request is ambiguous, or names a field or value that does not exist, the agent **asks one clarifying question** with options instead of guessing	Wrong SQL that runs is worse than a short question
S7	Optimize / debug / explain requests that arrive without SQL get a request for the SQL	Nothing to work on otherwise

Interpreting questions

#	Assumption	Example
I1	"After *month year*" means **on or after the first day** of that month, matching the brief's example (`hire_date > '2024-01-01'` for "after January 2024"). The interpretation is stated as an assumption in the answer	"hired after January 2024" → `HireDate >= '2024-01-01'` (135 rows)
I2	Relative dates are resolved against the **data snapshot date, 2026-10-20**, not the server clock	"last 90 days" → `date('2026-10-20', '-90 days')`
I3	Fiscal year = calendar year; quarters are calendar quarters	"Q1 2025" → Jan–Mar 2025
I4	**"Revenue" / "sales" means net revenue**: Σ quantity × sale price × (1 − discount) over orders not *Cancelled* or *Returned*, excluding shipping, unless the user asks otherwise	Metric definition in the knowledge base
I5	Sales figures use the **price at the time of sale** (`OrderItems.UnitPrice`), not today's list price (`Products.UnitPrice`)	"revenue by product"
I6	"Customers **from** X" filters where the customer lives (`Customers.State`); "orders **shipped to** X" filters the delivery state (`Orders.ShipState`)	~10% of orders ship to a different state
I7	"Sales rep" means the employee who booked the order (`Orders.EmployeeID`); "account manager" means `Customers.AccountManagerID`	Two different customer–employee links

#	Assumption	Example
I8	"Current employees" = Status *Active* or *On Leave*	Terminated employees keep their rows
I9	US regions follow the US Census definitions, plus common groupings (West Coast = CA, OR, WA; Pacific Northwest = WA, OR; New England, East Coast)	"West Coast customers" → `State IN ('CA','OR','WA')`
I10	Follow-ups **modify the previous query** unless they clearly start a new topic	"Only those from California" adds a filter to the previous SQL
I11	Sorting and limits are added only when the question implies them ("top 5", "latest", "most"); any default the model picks is listed in the answer's assumptions	"top customers"

Data and schema

#	Assumption	Reasoning
D1	The brief asks for "a sample database" without specifying one, so a realistic retail business (**Summit Supply Co.**) was designed: 5 tables, real join paths (including a self-reference and two different customer–employee links)	Multi-table questions are where SQL agents fail, while 5 tables stay small enough to understand at a glance
D2	**SQLite** is the execution engine; PostgreSQL and MySQL versions are produced by translation (sqlglot)	Zero-setup, file-based and easy to ship read-only. Other engines need only a different runner
D3	Data is **synthetic and deterministic** (fixed seed), with realistic patterns (seasonality, churn, never-ordered customers, never-sold products, status by order age, overdue invoices)	Reproducible answers, and edge cases that test the agent
D4	Dates are stored as ISO-8601 text; states as 2-letter USPS codes; discounts as fractions (0–0.30); booleans as 0/1 integers	Common real-world conventions that an agent must handle, all documented in the knowledge base
D5	The meaning of the data (descriptions, synonyms, value meanings, metrics, glossary) is **written by hand** in `semantic_layer.yaml`, and the build fails if it disagrees with the database	Column names alone are not enough to write correct business SQL

Safety

#	Assumption	Reasoning
X1	No generated SQL is trusted. Every query, from the model or the user, passes the deterministic validator, and the database connection is read-only regardless	Defence in depth; the model is never the last line of defence
X2	Prompt-injection attempts are **blocked before any model call** when they match known patterns, and user text is always treated as data in prompts	Cheaper and more reliable than asking the model to police itself
X3	Only one statement per request; stacked statements (`SELECT 1; DROP TABLE …`) are rejected	Stacked statements are the classic injection vector
X4	Result previews are capped at 500 rows, with the true total count shown; CSV export re-runs the validated SQL up to 50,000 rows; queries time out after 5 s	Protects the browser and the server

LLM and cost

#	Assumption	Reasoning
L1	The LLM is **Claude Haiku 5.5 on AWS Bedrock** for every step, with the router able to send any step to a larger model through configuration	Haiku is fast and inexpensive (≈ $0.0025 per question). The heavy lifting (grounding, validation) is deterministic, so a small model suffices
L2	Two tiers (fast/main) are kept even though both point to Haiku by default	Upgrading planning/SQL to Sonnet is a one-line `.env` change, with no code change
L3	Structured steps use the provider's native JSON-schema output, with a single tool-calling retry	Reliable parsing without regexes
L4	Costs shown in the developer view are computed from token counts × configured per-million rates (Haiku 5.5 list price by default)	Bedrock does not return a price; rates are configurable
L5	Explanations and suggestions are optional: if the model fails there, deterministic fallbacks are used and the SQL answer is still returned	The SQL is the primary deliverable

Users, sessions and limits

#	Assumption	Reasoning
U1	Reviewers get two **test logins** defined in the backend `.env` : a normal user and a developer	Lets reviewers compare the end-user experience with the internals. No sign-up flow is needed for an assignment
U2	The developer role sees internals (trace, plan, every LLM call, tokens, cost, run history, prompts, knowledge graph); this is enforced by the API, not just hidden in the UI	Internals can reveal prompts and schema details
U3	Chats are private to the account **and** browser (`username:client_id`), so several reviewers sharing one test account do not see each other's chats	Shared demo credentials
U4	Per-IP rate limits (6/min, 60/day) and login throttling protect the public demo and its LLM budget	A public link with shared credentials
U5	Conversation memory is stored in SQLite (checkpointer). On the free hosting plan the disk is ephemeral, so chats reset when the service restarts	Free hosting was preferred; with a persistent disk or managed database, chats survive restarts unchanged

Engineering decisions

Decision	Alternatives considered	Why this one
Plan → ground → SQL instead of question → SQL	Direct text-to-SQL with schema in the prompt	Grounding fixes value and join errors deterministically, and the plan is checkable
Deterministic validator + repair loop	Asking the model to self-check	Cheaper, reproducible, and gives precise fix hints to the repair step
Knowledge graph built from YAML + live profiling	Raw DDL only; vector search over docs	Exact values and join paths matter more than semantic similarity for 5 tables
Python-only backend (LangGraph, FastAPI)	Node/TypeScript backend	One language for the agent and the API; LangGraph's Python API is the reference
One file per graph node, folders by stage	One large module	Each node is readable and testable on its own; matches the intelligence-assistant layout
SSE for streaming	WebSockets	One-way stream, works through proxies, simple to consume
SQLite for app state and checkpoints	Postgres / Redis	Zero-ops for a single-instance demo; swappable
Scripted LLM in tests	Recorded cassettes / live calls	Deterministic, offline, and still exercises the real router and graph

Out of scope

- Write operations of any kind, schema changes, or stored procedures.
- Connecting to a user-supplied database at runtime (the knowledge base is built per schema).
- A golden-question accuracy benchmark (deliberately deferred).
- Horizontal scaling (the rate limiter and stores are single-instance by design).
- Docker packaging (the free hosting runs the app natively; not needed).